

异常和中断的支持

在前面的几章中，我们已经实现了一个支持 46 条 LoongArch 指令的流水线 CPU。不过，到目前为止，我们实现的都是指令系统的用户态部分。为了最终可以基于所实现的 CPU 搭建出一个小型的计算机系统，我们还要逐步添加指令系统中的特权态部分。这一章我们先介绍如何在 CPU 中支持**异常**（Exception）和**中断**（Interrupt）功能。

【本章学习目标】

- 理解异常和中断的软硬件协同处理机制。
- 理解精确异常的概念和处理方法。
- 掌握在流水线 CPU 中添加异常和中断支持的方法。

【本章实践目标】

本章有两个实践任务（见 7.5 节）。读者可以在学习本章内容的基础上完成这些任务。

7.1 异常和中断的基本概念

有关异常和中断的基本概念，读者可以参考《计算机体系结构基础》（第 3 版）的 3.2 节或其他资料学习。这里我们将根据 CPU 设计的需要梳理其中的关键概念。从 CPU 实现的角度来看，中断也是一种特殊的异常，所以在下文的表述中，除非专指中断，否则我们将统一用“异常”一词代指“异常”和“中断”。

7.1.1 异常是一套软硬件协同处理的机制

首先我们必须明确，异常是一套软硬件协同处理的机制。“异常”不是常态，其对应的情况发生频度不高，但处理起来比较复杂。本着“好钢用在刀刃上”的设计原则，我们希望尽可能由软件程序而不是硬件逻辑来处理这些复杂的异常情况。这样做既能保证硬件的设计复杂度得到控制，又能确保系统的实际运行性能没有太大的损失。

异常处理的绝大多数工作是由异常处理程序（软件）完成的，但是异常处理的开始和结束阶段必须要由硬件来完成。

异常处理的开始阶段

异常触发条件的判断由硬件完成，异常的类型、触发异常的指令的 PC 等供异常处理程序使用的信息由硬件自动保存。此外，硬件需要跳转到异常处理程序的入口执行，并保证跳转到异常入口后，处理器处于高特权等级。随后，异常处理程序接管后续处理过程。

异常处理的结束阶段

异常处理程序完成所有处理之后，需要返回发生异常的指令（或者一个事前指定的程序入口）处重新开始执行，这个过程除了要执行一次跳转外，还需要将处理器的特权等级调整回最初发生异常的指令所处运行环境的特权等级。

7.1.2 精确异常

为了在 CPU 中正确地实现异常功能，我们有必要强调一下“**精确异常**”这个概念。什么叫精确异常呢？它要达到的效果是：当系统软件处理完异常返回后，对于发生异常的指令和它后面的指令，就好像异常没有发生过一样。这里的“前后”是根据程序中定义的顺序来判定的。

在上面这个表述中，是让谁觉得“像没有发生过异常一样”？是指令，所以说，内容没有发生变化是从指令的视角来看的。这里我们用“看”这个拟人的修辞方式来表述，虽然生动，但从概念的角度难免不够精准。我们还需要再唠叨几句。

指令能“看”到的内容有什么？它只能看到 ISA 中定义的那些内容，看不到那些不包含在 ISA 定义范畴内的微结构层面的内容。举个例子来说，PC、通用寄存器、内存、处理器所处特权等级属于指令能看到的，而处理器中每一级流水的状态则属于指令看不到的。

指令又是怎么“看”的呢？这个过程的严谨而全面的表述抽象且冗长，这里我们表

述得稍微工程化一些，通过几个典型的例子来说明。

【例一】 如果一条指令有寄存器的源操作数，那么它在 CPU 流水线中要到译码级生成源操作数的时候才开始“看”到这个寄存器。

【例二】 如果是一条 load 指令，那么它在 CPU 流水线中直到访存级^①才开始“看”到内存这个地址。

【例三】 如果是一条特权指令，即它只有在 CPU 处于特权状态下才能执行，假设有关特权指令合法性的检查是在译码级进行的，那么这条指令直到译码级才能“看”到 CPU 的特权等级状态。

【例四】 假设某些地址空间只有特权态的程序才能访问，那么任何指令必须在取指阶段发起访存请求的时候就“看”到 CPU 的特权等级状态。

从这四个例子中，读者多少能体会到指令的“看”在一个 CPU 中是如何进行的。之所以需要关注这个过程，是因为仅从程序员的角度来看，每条指令对处理器的状态更新都应该是原子的、瞬时完成的，但是程序在处理器中真实运行的时候，一条指令涉及的处理器的状态的更新却是分布的、有延迟的。作为处理器的设计人员，我们需要用这个分布的、有延迟的真实处理器给软件人员构造出一个原子的、瞬时的抽象处理器。

7.2 LoongArch 指令系统中与异常相关的功能定义

在了解了异常的一般性概念的基础之上，我们再来具体分析 LoongArch 指令系统中与异常相关的功能定义，以形成我们的最终设计。

7.2.1 控制状态寄存器

前面提到，异常是一个软硬件协同的处理过程。在这个过程中，硬件逻辑电路和异常处理软件需要进行必要的信息交互。为了实现精确异常，这些交互的信息不能放在用户态程序可见的软件上下文中。（请读者思考一下为什么。）LoongArch 指令系统中定义了一组独立的寄存器用于这类信息的交互，统称为**控制状态寄存器**（Control Status Register，CSR）。

① 也许读者会说执行级发出数据 RAM 读请求的时候才是最早“看”到该地址内存值的时机。如果 load 指令在访存级获得访存结果只来自数据 RAM 的 Q 端输出，那么这种看法就是完全正确的。如果 load 指令在访存级还有可能获得 store 指令直接前递过来的数值，那么正文中的表述就更为合适。正文中给出的是一种在大多数情况下都正确的表述。

就本章中涉及异常和中断的处理而言，相关的 CSR 有：CRMD、PRMD、ECFG、ESTAT、ERA、BADV、EENTRY、SAVE0 ~ 3、TID、TCFG、TVAL、TICLR。这些寄存器的详细定义请参看指令手册第 7 章。初次接触 LoongArch 指令系统的读者在学习这部分内容时，建议首先快速浏览一遍指令手册第 7 章中相关寄存器的内容，这一遍可以“不求甚解”，对几个寄存器有个初步印象即可。然后阅读指令手册第 6 章熟悉处理器硬件响应异常的一般性处理过程以及中断处理的相关内容。最后在敲定设计方案时，如果遇到与 CSR 相关的内容，再仔细查阅指令手册第 7 章的内容，掌握具体细节。

此外补充说明一点，指令手册中 CSR 定义描述中的“读写”属性是从软件程序的视角来说明的，与具体硬件实现时这些寄存器的读、写属性有联系，但并不是直接对应的。硬件实现时所看到的读写属性在本章后面进行设计细节讲解时将会具体讲述。

7.2.2 异常产生条件的判定

本章需要实现的异常包括中断 INT、取指地址错 ADEF、地址非对齐 ALE、系统调用 SYS、断点 BRK 和指令不存在 INE，共计 6 种。考虑到除了中断之外，指令手册中其他异常产生条件分散在可能与之相关的各指令定义处，为了方便读者理解，这里对各异常产生条件的判定进行总结。

7.2.2.1 处理器核内部判定接收到中断的过程

注意，请将本小节内容与指令手册 6.1 节、7.4.4 节和 7.4.5 节的内容结合起来阅读。

根据 LoongArch32 精简版指令集的定义，每个处理器核内部可以记录 12 个线中断。除去应用于多核场景下的核间中断目前暂不考虑，还包括 8 个硬中断、2 个软中断和 1 个定时器中断。

硬件中断的源头来自处理器核外部，读者可以认为每个处理器核都有 8 个中断输入引脚，设备或中断控制器将高电平有效的中断信号连接到这 8 个输入引脚上，而处理器核内部 ESTAT 控制状态寄存器 IS（Interrupt State）域的 9..2 这 8 位（RTL 上对应 8 个触发器）则直接对中断输入引脚的信号采样。软件中断顾名思义是由软件来设置的，通过 CSR 写指令对 ESTAT 状态控制寄存器 IS 域的 1..0 这两位写 1 或写 0 就可以完成两个软件中断的置起和撤销。定时器中断的状态记录在 ESTAT 控制状态寄存器 IS 域的第 11 位。（在后面的 7.2.2.2 节将有关于定时器中断的更进一步介绍。）

ESTAT 状态控制寄存器 IS 的 11 和 9..0 位记录了上述 11 个中断的状态，且均是高电平表示有效。不过，处理器核内部判断是否接收到中断不仅要看这些位中是否存在有

效值，还要看中断的使能情况。中断的使能情况分两个层次：低层次是与各中断一一对应的局部中断使能，通过 ECFG 控制寄存器的 LIE（Local Interrupt Enable）域的 11 和 9..0 位来控制；高层次是全局中断使能，通过 CRMD 控制状态寄存器的 IE（Interrupt Enable）位来控制。

综合上述情况，处理器核内部判定接收到中断的标志信号 `has_int` 可以实现为：

```
assign has_int = ((csr_estat_is[12:0] & csr_ecfg_lie[12:0]) != 13'b0) && (csr_crmd_ie == 1'b1);
```

很显然无论是接收到一个外部中断还是多个外部中断，`has_int` 信号都可以置为有效，即 CPU 硬件并不考虑这其中的细节差异^①。当确实同时接收到多个中断时，后续的处理交给软件的中断处理函数进行。

7.2.2.2 核内定时器中断产生过程

定时器中断经常用于操作系统的调度和计时功能的实现。该中断源来自定时器，通常分为核外和核内两种实现方式。LoongArch 指令系统采用了核内实现定时器中断源的方式。简单来说，在每个 LoongArch32 处理器核内部实现一个 32 位的计数器，在开启定时功能后每个时钟周期减 1，当减到 0 值即可触发一次定时器中断。接下来介绍其定义细节：

- 1) 定时器的软件配置集中在 TCFG（Timer Config）控制状态寄存器，包括它的启动使能、倒计时初始值和倒计时模式。其中倒计时模式分为两种，一种是减到 0 后即停止计数，另一种是减到 0 后自动装载倒计时初始值再次开始新一轮倒计时。
- 2) 定时器与 `rdcntv{l/h}.w` 指令所访问的计时器使用同一时钟，这个时钟要求频率固定。在本书的实践任务范围内，我们尚不考虑处理器核的变频和关停，所以可以采用处理器核流水线的时钟。
- 3) 定时器当前的计数值仅可以通过读取 TVAL 状态寄存器近似^②获得，它与 `rdcntv{l/h}.w` 指令读取的计时器值来自两个截然不同的对象。
- 4) 当定时器倒计时到 0 时硬件将 ESTAT 控制状态寄存器 IS 域的第 11 位置 1，软件通过把 TICLR 控制寄存器的 CLR 位写 1 将 ESTAT 控制状态寄存器 IS 域的

① 仅在 LoongArch 精简版才是这样处理的，LoongArch 商用版本中硬件可以按照固定优先级处理同时接收到的多个中断。

② 这里说“近似”是因为定时器的每个时钟周期都在自减 1，软件用来读取 TVAL 寄存器的 CSR 读指令的执行需要多个时钟周期，所以它只能反映执行这条 CSR 指令的这段时间中某个时钟周期的定时器数值。

第 11 位清 0。请读者特别注意这一句描述的措辞。它不是“ESTAT 控制状态寄存器 IS 域的第 11 位一直采样定时器数值是否等于 0 的判定结果”，也没有说“将定时器数值置为非 0 值即可将 ESTAT 的 IS 域第 11 位清 0”。前面引号里面的两段描述是不对的，部分初学者容易想当然地将指令手册的描述理解成这样。如果读者还是不懂前面加粗的话，那么我们再来看下面的代码对比：

```
always @(posedge clock) begin
    csr_estat_is[11] <= (timer_cnt[31:0]==32'b0);
end
```

上面的写法就是基于错误认识写出的代码，而下面的代码才是正确的。

```
always @(posedge clock) begin
    if (csr_tcfg_en && timer_cnt[31:0]==32'b0)
        csr_estat_is[11] <= 1'b1;
    else if (csr_we && csr_num==`CSR_TICLR
            && csr_wmask[`CSR_TICLR_CLR]
            && csr_wvalue[`CSR_TICLR_CLR])
        csr_estat_is[11] <= 1'b0;
end
```

请读者对比上面两段代码，仔细体会一下其中的差别，然后再想一想，为什么定时器中断状态位的设置和清除要定义得这么“绕”？（提示两个关键词：**脉冲信号**、**电平中断**。）

此外，有经验的读者可能会从上面第二段正确的代码中看出一个小问题：如果 if 和 else if 的条件同时有效怎么办？是的，理论上这种情况是存在的。我们给出的代码其实是在考虑了这种情况之后才特意将置 1 调整为更高的优先级，其出发点是——尽量不漏掉中断。不过，硬件方面也只能努力到这个程度了，相信读者很容易举出其他定时器中断被漏掉的场景。其实，只有在软件考虑不周的情况下，才有可能出现这种一次定时器中断的软件处理过程“慢”到下一轮（甚至是更晚的轮次）定时器中断都来了，软件才执行到清除当前定时器中断状态位的不合理场景。

7.2.2.3 取指地址错异常 ADEF 判定条件

LoongArch 指令系统要求所有指令的 PC 都是字对齐的（地址最低两位为全 0），当违反这一规定时，将触发取指地址错误异常。错误的 PC 值将被硬件记录在 BADV 控制状态寄存器中。此时记录异常返回地址的 ERA 控制状态寄存器中应记录出错的 PC。不过，由于取指地址错误异常处理结束后通常不会直接返回到出错的 PC 处继续执行，所以此时 ERA 控制状态寄存器中记录的信息意义不大，软件在诊断过程中通常将读取

BADV 状态控制寄存器中的信息。

7.2.2.4 地址非对齐异常 ALE 判定条件

地址非对齐异常仅针对 load、store 这类访存指令。当 ld.h、ld.hu 和 st.h 指令的地址最低位不为 0 时，或者当 ld.w、st.w、ll.w^①和 sc.w 指令的最低两位不为全 0 时，触发地址非对齐异常。此时出错的访存“虚”地址将被硬件记录在 BADV 控制状态寄存器中，其余按照异常的一般处理过程进行处理。

7.2.2.5 指令不存在异常 INE 判定条件

当取回的指令码不属于任何一条已实现^②的指令时，将触发指令不存在异常，余下的按照异常的一般处理过程进行处理。

7.2.2.6 系统调用 SYS 和断点异常 BRK 判定条件

当执行 syscall 指令时触发系统调用异常，当执行 break 指令时触发断点异常，余下的按照异常的一般处理过程进行处理。

7.2.3 响应异常后硬件的一般处理过程

除了前面具体指出的特殊处理操作外，处理器响应异常后硬件的一般处理过程在指令手册的 6.2.3 节已有明确定义，请读者自行参看。提醒读者，有关异常入口的定义在指令手册 6.2.1 节有明确定义。

7.2.4 异常处理返回指令

7.1 节中提到了在异常处理的结束阶段，异常处理程序要完成两个操作：一是回到异常出现的位置，二是恢复出现异常时的特权等级。这两个操作需要同步完成，即它们最好通过一条指令完成。在 LoongArch 指令系统规范中，定义了 ERTN 指令来原子地^③完成这两个操作。ERTN 指令一方面将 ERA 寄存器中存放的指针值作为目标地址跳转过去，同时将 PRMD 控制状态寄存器中的 PPLV 和 PIE 域的值分别写入 CRMD 控制状态寄存器的 PLV 和 IE 域中。

① ll.w 和 sc.w 指令截至目前尚未实现，但仍列举在此处以便让读者建立正确的印象。

② 严格来说是指令集中已定义的。这里采用这样的描述是为了配合本书中的进阶实践进度，否则译码部件需要按照 LoongArch32 位精简版指令集定义的所有指令进行译码判断。

③ 这里的原子性是指从软件视角来看已不再可分。对软件程序而言，一条机器指令已经是一个不可再分的最小执行单元了。

7.2.5 CSR 读写指令

LoongArch 指令系统中并没有定义直接操作 CSR 的算术逻辑运算类指令，而是定义了三条 CSR 读写指令用于在通用寄存器和 CSR 之间交互数据。这三条指令的具体定义请参看指令手册的 4.2.1 节。需要提醒初学者的是，`csrwr` 指令会将写入 CSR 的旧值写入到第 `rd` 项通用寄存器中，事实上，实现者可以把 `csrwr` 指令理解为一个写掩码固定为全 1 的 `csrxchg` 指令。为什么要定义这种 CSR 读写指令以及 CSR 读写指令如何使用的具体举例，感兴趣的读者可以阅读《计算机体系结构基础》(第 3 版) 的第 3 章。

7.3 流水线 CPU 实现异常和中断的设计要点

在了解了 LoongArch 指令系统关于异常和中断的具体规定之后，我们接下来进一步从处理器微结构设计的视角来分析相关的设计要点。

7.3.1 异常检测逻辑的实现

通过前面的基本概念梳理，我们已经知道异常发生条件的检测是由硬件完成的。因此，首先我们来逐一考虑各个异常的检测逻辑该如何生成。

7.3.1.1 取指地址错异常检测逻辑

通过对 7.2.2.3 节的学习，可知需对取指所用的 PC 的最低两位进行判断，如果不是 2'b00 的话，则置起取指地址错异常标志。我们推荐在 `pre-IF` 级就进行这一判断。从严格意义上讲，出现异常的取指地址不应该用来发起取指的请求，因为此时这个 PC 可能已经完全不正确，所以它的访存行为也不在软件人员的预想之内，最严重时可能导致死机等错误。不过截至目前的实践任务中，我们只从指令 RAM 中取指令（意味着任何取指请求都只被指令 RAM 响应），而且访问地址已经将 PC 的最低两位抹为全 0，所以即使将检测到存在取指地址错异常的 PC 作为取指请求发出也不会造成无法恢复的负面效果。我们在这里说的这些内容，算是埋下一个伏笔，本书后面讲到 AXI 总线接口实现的时候还会再谈及这个初学者容易忽视的设计要点。

7.3.1.2 地址非对齐异常检测逻辑

通过对 7.2.2.4 节的学习，可知需要对 `load`、`store` 指令的地址进行判断，当访存地址出现非对齐情况时，则置起地址非对齐异常标志。与前一节的设计思路一致，我们推

荐在发起访存请求的 EXE 级进行上述判断，这样可以在发现异常情况的时候停止用错误地址发起访存请求。

7.3.1.3 指令不存在异常检测逻辑

通过对 7.2.2.5 节的学习，可知指令不存在异常需要对指令译码后才能得到检测结果。由于我们需要在译码阶段对每一条实现的指令进行解码生成控制信号，因此自然就可得到“不是任何一条实现的指令”的条件。

7.3.1.4 系统调用和断点异常检测逻辑

通过对 7.2.2.6 节的学习，可知系统调用和断点异常的检测是最直接的，即译码发现是 `syscall` 指令就置起系统调用异常标志，译码发现 `break` 指令就置起断点异常标志。

7.3.1.5 中断异常检测逻辑

中断异常检测首先要在处理器核内产生“接收到中断”这样一个标志信号，其判定方式在 7.2.2.1 节已有介绍。至于在处理器核内部如何产生定时器中断，请读者回顾一下 7.2.2.2 节的内容。这一小节重点讲述接收到的中断信号如何被标记到指令上这方面的设计。

我们会发现，前面所有的异常产生是指令（或程序）自身的属性造成的，而中断是由外部事件触发的，它与指令之间并无直接对应关系。既然我们将中断视作一种特殊的异常，那就意味着我们希望能通过一套异常的处理框架，同时处理非中断和中断这两种属性的异常。我们采取的设计方法是，将异步的中断事件动态地标记在某一条指令上，被标记的指令随后就被赋予了中断这种异常，那么随后的处理就和其他异常非常类似了。那么五级流水线 CPU 中同一时刻可能会有多条指令，将中断标记在哪一条指令上合适呢？理论上讲，任一级流水线上的指令都可以选出来被标记上中断异常。出于精确异常实现开销和中断响应延迟两方面的折中，我们通常将中断标记在译码级的指令上。但请了解这并不是唯一的设计方案。

有些心细的读者可能会问，既然 LoongArch 只能接收电平中断输入，那么 CPU 内部生成的是否有中断的信号也会维持很多拍，会不会将很多指令都标记上中断异常，这是否会导致什么问题？答案是不会有问题。首先，对于被中断异常打断的程序段来说，即使出现了多条指令被标记上中断异常，但是根据接下来会讲到的精确异常实现的需要，只有程序顺序在最前面的那条指令才能报出中断异常，而程序顺序在它后面的那些指令会被取消掉，不会出现一个中断被多次报出来的情况。至于说硬件报出中

断异常，进入到中断异常处理程序之后，CRMD 控制状态寄存器的 IE 位已被硬件置为 0（请参看指令手册 7.4.1 节），此时处理器核内部“接收到中断”的信号就不会再有效了。

7.3.2 精确异常的实现

我们从指令系统规范的定义中知道，异常发生之后，处理器硬件需要设置一些控制状态寄存器、进入最高特权等级并跳转到相应的异常入口。同时，我们通过 7.3.1 节的分析也了解到不同类型的异常检测逻辑可以出现在处理器核的不同流水级。那么是不是一旦发生异常，处理器就要立即产生诸如修改控制状态寄存器、跳转到异常处理入口的动作呢？如果这样处理，那么这些控制状态寄存器和取指 PC 因异常而更新的来源就会有多个，而同一时刻我们又只能选择一个来源对其进行更新，于是我们又会碰到如何选择的问题。事实上，为了实现精确异常，我们发现并不需要一发生异常就着急地修改控制状态寄存器和取指 PC。

回顾一下 7.1.2 节中关于精确异常的分析，我们发现，发生异常时仅需要考虑如何处理那些在流水线中的指令。因为很显然，对于那些程序顺序在发生异常指令之前的指令，如果它们都已经执行完毕并退出流水线，那么必然都已经产生了执行效果；而那些程序顺序在发生异常指令之后且还没有取进流水线的指令，等到它们真正取进流水线的时候，一定是在异常处理返回之后了。这些指令自然都遵循精确异常的语义。

那么发生异常时，那些已经在流水线中的指令该如何处理呢？具体的方法有很多种，这里介绍一种常用的设计思路：**异常发生的判断逻辑分布在各流水级，靠近与之相关的数据通路；发现异常后将异常信息附着在指令上沿流水线一路携带下去，直至写回级才真正报出异常，此时才会根据所携带的异常信息更新控制状态寄存器；写回指令报出异常的同时，清空所有流水级缓存的状态，并将 nextPC 置为异常入口地址。**当然，报出异常的流水级不一定要设定为最后一级（写回级），设定在执行级或访存级也可以，这样还会减少设计负担。但是要注意：**选择报出异常的流水级的原则是，在该级之后的流水级不能产生新的异常（比如报出异常的流水级设定为执行级，则要求访存级和写回级不能产生或标记上新的异常），否则就违反了精确异常的要求。**

简单分析一下上面这种做法的合理性。发生异常的指令到达写回级的时候，程序序在它前面的指令都已经退出流水线了，所以这些指令的执行效果都已经产生。写回级指令报异常时清空所有流水线，意味着那些已经进入 CPU 流水线、在异常指令之后（含异常指令）的指令**都不会**产生执行效果。异常入口地址最早是在报异常的下一拍才能进

入 CPU 流水线, 所以它“看”到的处理器状态都是更新完成的状态。

上面的描述中, 之所以将“都不会”三个字加粗着重标记, 是因为这样的特性并不是显然可得。虽然所有对通用寄存器的写都放在写回级进行, 但是 store 类指令对数据 RAM 的写命令也是在执行级就发出了。因此, 如果当前拍写回级有指令要报异常而访存级是一条 store 指令, 则上一拍若没有在执行级对这些指令做任何处理, 那么在报出异常的时候, 内存就已经被异常之后的指令修改了, 这就违反了精确异常。那么该如何处理呢? 现阶段一种简单有效的方式是: store 指令若想在执行级发出写命令, 那么需要检查当前访存级和写回级上是否存在已标记为异常或可能标记为异常的指令, 也要检查自己有没有产生异常或标记异常。庆幸的是, 就目前支持的指令和异常类型来说, 指令在访存级和写回级不会再判断出新的异常了。也就是说, 位于执行级的 store 指令只需要检查当前访存级和写回级上有没有已标记为异常的指令就可以了, 当然, 它也会在执行级检查自己有没有被标记上异常。这种情况下, 异常信息都保存在流水线缓存触发器中, 所以不会对数据 RAM 的写使能信号的时序造成特别大的影响。在真正实用的处理器中, store 指令不会在执行级就真的发出可修改内存的命令, 而是要等到 store 指令从写回级执行完毕之后才真正发出修改内存的动作。这套功能通常需要 store buffer 或 store queue 这样的结构来支持。

7.3.3 控制状态寄存器的实现

本章前面的分析过程连续出现了涉及控制状态寄存器操作的内容。考虑到这部分实现在一般的教科书中很少涉及, 而不少初学者在实现时也有的一种无从下手的感觉, 因此在这里专门介绍其实现过程需要注意的一些问题。

控制状态寄存器与大家一般学习到的通用寄存器、浮点寄存器有一个最大的区别就是, 它不仅能被软件用指令直接读、写, 而且能够被硬件直接更新或是直接控制硬件的行为。尽管说指令手册的第 7 章对各个控制状态寄存器集中进行了描述, 不过要理解其每个域的工作机制, 尤其是其与处理器硬件如何交互控制和状态信息, 要从特权等级、异常、内存管理等指令集核心态部分的工作原理入手, 厘清处理过程中软硬件是如何协同工作的。

7.3.3.1 控制状态寄存器实现代码的组织形式

控制状态寄存器要被 csrrd、csrwr、csrxchg 这样的 CSR 指令访问, 又要和处理器核中各级流水线以及接口交互, 前一个需求倾向于集中实现, 而后一个需求似乎又倾向于分散实现, 那么具体实现时代码该如何组织才合适呢? 尽管最合理的回答是: 该集中

就集中，该分散就分散，视情况而定。相信初学者看着这种建议不免腹诽，因此我们给出明确的建议如下：

- 1) 把所有控制状态寄存器集中到一个模块中实现；
- 2) 模块接口分为用于指令访问的接口和与处理器核内部硬件电路逻辑直接交互的控制、状态信号接口两类；
- 3) 指令访问接口包含读使能 (`csr_re`)[⊖]、寄存器号 (`csr_num`)、寄存器读返回值 (`csr_rvalue`)、写使能 (`csr_we`)、写掩码 (`csr_wmask`) 和写数据 (`csr_wvalue`)；
- 4) 与硬件电路逻辑直接交互的接口信号视需要各自独立定义，无须再统一编码，如送往预取指 (`pre-IF`) 流水级的异常处理入口地址 (`ex_entry`)、送往译码流水级的中断有效信号 (`has_int`)、来自写回流水级的 `ertn` 指令执行的有效信号 (`ertn_flush`)、来自写回流水级的异常处理触发信号 (`wb_ex`) 以及异常类型 (`wb_ecode` 和 `wb_esubcode`) 等。

上面的建议虽然未必是最优雅的实现方案，但至少界面清晰，便于增量式开发过程中的代码维护和错误定位。至于说控制状态寄存器模块是实例化在某一级流水线模块内部还是与各流水级模块处于并列地位，并无严格限定，尽管我们倾向于后一种方式。

7.3.3.2 控制状态寄存器读、写来源梳理

根据前面的介绍，控制状态寄存器模块中将实现各个 CSR。通过分析指令集中关于 CSR 的定义，我们将发现一个特点：单个 CSR 可能包含多个域，而且同一个 CSR 中不同域的维护规则可能不一样。基于这个发现，我们建议读者将 CSR 中的各个域作为代码实现的基本单位，而不要以一个 CSR 整体作为基本单位。下面将对 CSR 的不同域逐个分析并给出实现举例。

1. CRMD 的 PLV 域

从指令手册的定义可知 CRMD 的 PLV 域可以通过 CSR 指令更新，而且在触发异常和 `ertn` 指令执行时也将被更新。那么它的 Verilog 代码可以实现如下：

```
always @(posedge clock) begin
    if (reset)
        csr_crmd_plv <= 2'b0;
    else if (wb_ex)
        csr_crmd_plv <= 2'b0;
```

⊖ 本书后面的实现建议都是采用类似寄存器堆读取的异步读，所以读使能实际上可以省略。

```

    else if (ertn_flush)
        csr_crmd_plv <= csr_prmd_pplv;
    else if (csr_we && csr_num==`CSR_CRMD)
        csr_crmd_plv <= csr_wmask[`CSR_CRMD_PLV]&csr_wvalue[`CSR_CRMD_PLV]
            | ~csr_wmask[`CSR_CRMD_PLV]&csr_crmd_plv;
end

```

上面的实现中表示复位时需要将 CRMD 的 PLV 域置为全 0 (最高优先级), 这是在指令手册的 6.3 节定义的。另外, 在被 CSR 写操作 (对应 `csrwr` 和 `csrxchg` 指令) 更新时, 需要考虑写掩码。这段代码应该比较好理解, 每一个写入条件和写入的值基本上都可以与指令手册中的定义一一对应。实现时, 重点关注 CSR 模块的各个输入信号没有接错即可。

不过, 上面这种代码风格虽然很直观, 但它有一个最容易引入出错的风险点, 那就是 “`if...else if...else if`” 形式上是存在优先级的。这就意味着, 你不能只盯着单个 `if` 后面的条件来检查它是否会被正确执行。举个例子来说, 如果代码是 “`if (cond_A) do_A else if (cond_B) do_B else if (cond_C) do_C`”, 那么 “`do_C`” 这个操作发生的条件实际上是 “`!cond_A && !cond_B && cond_C`”, 而不是简单的 “`cond_C`”。

2. CRMD 的 IE 域

从指令手册的定义可知 CRMD 的 IE 域维护与 PLV 域很相似, 其 Verilog 代码可以实现如下:

```

always @(posedge clock) begin
    if (reset)
        csr_crmd_ie <= 1'b0;
    else if (wb_ex)
        csr_crmd_ie <= 1'b0;
    else if (ertn_flush)
        csr_crmd_ie <= csr_prmd_pie;
    else if (csr_we && csr_num==`CSR_CRMD)
        csr_crmd_ie <= csr_wmask[`CSR_CRMD_PIE]&csr_wvalue[`CSR_CRMD_PIE]
            | ~csr_wmask[`CSR_CRMD_PIE]&csr_crmd_ie;
end

```

对比 CRMD 的 IE 域和 PLV 域的代码, 可以发现各 `if` 分支的条件是一样的, 而且各分支下的操作处理代码也很相似, 那么这种情况下, 最好将一个 CSR 的多个域的赋值集中到一个 `always` 块中。

3. CRMD 的 DA、PG、DATF、DATM 域

目前我们设计的处理器还没有实现 MMU 的全部功能, 仅支持直接地址翻译模式,

所以 CRMD 的 DA、PG、DATF、DATM 域可以暂时置为常值。等到第 9 章的实践任务中可完善 DA 和 PG 域的功能，等到第 10 章的实践任务中可进一步完善 DATF 和 DATM 域的功能。

```
assign csr_crmd_da   = 1'b1;
assign csr_crmd_pg   = 1'b0;
assign csr_crmd_datf = 2'b00;
assign csr_crmd_datm = 2'b00;
```

4. PRMD 的 PPLV、PIE 域

分析指令手册中的定义，可知 PRMD 的 PPLV 和 PIE 域的维护很相似，所以将其 Verilog 代码实现如下：

```
always @(posedge clock) begin
    if (wb_ex) begin
        csr_prmd_pplv <= csr_crmd_plv;
        csr_prmd_pie  <= csr_crmd_ie;
    end
    else if (csr_we && csr_num==`CSR_PRMD) begin
        csr_prmd_pplv <= csr_wmask[`CSR_PRMD_PPLV]&csr_wvalue[`CSR_PRMD_PPLV]
            | ~csr_wmask[`CSR_PRMD_PPLV]&csr_prmd_pplv;
        csr_prmd_pie  <= csr_wmask[`CSR_PRMD_PIE]&csr_wvalue[`CSR_PRMD_PIE]
            | ~csr_wmask[`CSR_PRMD_PIE]&csr_prmd_pie;
    end
end
```

有的读者看了上述代码后可能会问为什么没有复位有效时置初值的逻辑。由于指令手册中并未定义上述 CSR 域需要进行复位，所以将代码写成上面这样仍是符合指令集规范的。换言之，是由软件人员来保证在读取这些值之前一定发生过对这些 CSR 域的更新。譬如，处理器核复位后，如果还没有发生过任何异常或者软件还没有设置 PRMD 的 PPLV 和 PIE 域，是不能执行 ertn 指令的。

5. ECFG 的 LIE 域

从指令手册的定义可知 ECFG 的 LIE 域仅会被 CSR 指令更新。其 Verilog 代码实现如下：

```
always @(posedge clock) begin
    if (reset)
        csr_ecfg_lie <= 13'b0;
    else if (csr_we && csr_num==`CSR_ECFG)
        csr_ecfg_lie <= csr_wmask[`CSR_ECFG_LIE]&13'h1bff&csr_wvalue[`CSR_ECFG_LIE]
            | ~csr_wmask[`CSR_ECFG_LIE]&13'h1bff&csr_ecfg_lie;
    end
```


6. ESTAT 的 IS 域

ESTAT 的 IS 域中, 1..0 位、9..2 位、11 位、12 位的更新来源存在区别, 其依次仅被 CSR 指令更新、仅通过采样处理器核硬件中断输入引脚更新、仅根据定时器计数器和 TICLR.CLR 域的写更新、仅通过采样处理器核的核间中断输入引脚更新。第 10 位没有定义, 保险起见我们将其恒置为 0。其 Verilog 代码实现如下:

```
always @(posedge clock) begin
    if (reset)
        csr_estat_is[1:0] <= 2'b0;
    else if (csr_we && csr_num==`CSR_ESTAT)
        csr_estat_is[1:0] <= csr_wmask[`CSR_ESTAT_IS10]&csr_wvalue[`CSR_ESTAT_
            IS10]
            | ~csr_wmask[`CSR_ESTAT_IS10]&csr_estat_is[1:0];

    csr_estat_is[9:2] <= hw_int_in[7:0];

    csr_estat_is[10] <= 1'b0;

    if (timer_cnt[31:0]==32'b0)
        csr_estat_is[11] <= 1'b1;
    else if (csr_we && csr_num==`CSR_TICLR && csr_wmask[`CSR_TICLR_CLR]
        && csr_wvalue[`CSR_TICLR_CLR])
        csr_estat_is[11] <= 1'b0;

    csr_estat_is[12] <= ipi_int_in;
end
```

有关 IS 域第 11 位的设计在 7.2.2.2 节已经讨论过, 这里请对比看一下 IS 域的 1..0 位和 9..2 位的更新逻辑, 并与指令手册中两者的读写属性定义对照着分析一下, 加深对 CSR 寄存器域读写属性“RW”和“R”的理解。

7. ESTAT 的 Ecode 和 EsubCode 域

ESTAT 的 Ecode 和 EsubCode 域需要在触发异常时填入异常的类型代号。前面在讲述精确异常的实现时, 提到了在流水线各相关处进行异常检测, 生成发生异常的标志信号后随流水线逐级传递直至写回级再触发异常。那么, 代码编写时沿流水线逐级传递的是每个异常一个单独的标志信号, 还是已经根据指令手册定义进行编码之后的 Ecode 和 EsubCode 值呢? 对初学者来说, 我们推荐采用每个异常单独一个标志信号的传递方式, 最后在写回级编码为 Ecode 和 EsubCode 值送到 CSR 模块。这样代码的可阅读性是最容易保证的。按照这一思路, 得到的 Verilog 代码实现如下:

```
always @(posedge clock) begin
    if (wb_ex) begin
```

```

        csr_estat_ecode    <= wb_ecode;
        csr_estat_esubcode <= wb_esubcode;
    end
end

```

8. ERA 的 PC 域

当位于写回级指令触发异常时，需要记录到 ERA 寄存器的 PC 就是当前写回级的 PC。其 Verilog 代码实现如下：

```

always @(posedge clock) begin
    if (wb_ex)
        csr_era_pc <= wb_pc;
    else if (csr_we && csr_num==`CSR_ERA)
        csr_era_pc <= csr_wmask[`CSR_ERA_PC]&csr_wvalue[`CSR_ERA_PC]
            | ~csr_wmask[`CSR_ERA_PC]&csr_era_pc;
    end
end

```

9. BADV 的 VAddr 域

BADV 的 VAddr 域和 ERA 的 PC 域的维护有相似之处，都是在写回级指令触发异常时，记录该指令的一些信息。这里需要注意一点，在支持异常处理之前，处理器流水线中在访存级和写回级是不需要保存 load、store 指令完整的虚地址的。那么此处为了正确维护 BADV 的 VAddr 域，我们就需要在执行级、访存级和写回级增加与之对应的数据通路。至于说在访存级和写回级的流水线缓存中是新增一个 VAddr 域还是复用其他域，只是一个具体实现的小细节，读者可以凭自己喜好选择实现方案。这里给出 Verilog 代码实现如下：

```

assign wb_ex_addr_err = wb_ecode==`ECODE_ADE || wb_ecode==`ECODE_ALE;

always @(posedge clock) begin
    if (wb_ex && wb_ex_addr_err)
        csr_badv_vaddr <= (wb_ecode==`ECODE_ADE &&
            wb_esubcode==`ESUBCODE_ADEF) ? wb_pc : wb_vaddr;
    end
end

```

10. EENTRY 的 VA 域

EENTRY 的 VA 域仅能被 CSR 指令更新，其实现比较简单，示例 Verilog 代码如下：

```

always @(posedge clock) begin
    if (csr_we && csr_num==`CSR_EENTRY)
        csr_eentry_va <= csr_wmask[`CSR_EENTRY_VA]&csr_wvalue[`CSR_EENTRY_VA]
            | ~csr_wmask[`CSR_EENTRY_VA]&csr_eentry_va;
    end
end

```

11. SAVE0~3

SAVE0 ~ 3 就是提供给特权态软件临时存放数值用的, 其实现比较简单, 示例 Verilog 代码如下:

```
always @(posedge clock) begin
    if (csr_we && csr_num==`CSR_SAVE0)
        csr_save0_data <= csr_wmask[`CSR_SAVE_DATA]&csr_wvalue[`CSR_SAVE_DATA]
            | ~csr_wmask[`CSR_SAVE_DATA]&csr_save0_data;
    if (csr_we && csr_num==`CSR_SAVE1)
        csr_save1_data <= csr_wmask[`CSR_SAVE_DATA]&csr_wvalue[`CSR_SAVE_DATA]
            | ~csr_wmask[`CSR_SAVE_DATA]&csr_save1_data;
    if (csr_we && csr_num==`CSR_SAVE2)
        csr_save2_data <= csr_wmask[`CSR_SAVE_DATA]&csr_wvalue[`CSR_SAVE_DATA]
            | ~csr_wmask[`CSR_SAVE_DATA]&csr_save2_data;
    if (csr_we && csr_num==`CSR_SAVE3)
        csr_save3_data <= csr_wmask[`CSR_SAVE_DATA]&csr_wvalue[`CSR_SAVE_DATA]
            | ~csr_wmask[`CSR_SAVE_DATA]&csr_save3_data;
end
```

12. TID

TID 寄存器的维护也比较简单, 其示例 Verilog 代码如下:

```
always @(posedge clock) begin
    if (reset)
        csr_tid_tid <= coreid_in;
    else if (csr_we && csr_num==`CSR_TID)
        csr_tid_tid <= csr_wmask[`CSR_TID_TID]&csr_wvalue[`CSR_TID_TID]
            | ~csr_wmask[`CSR_TID_TID]&csr_tid_tid;
end
```

13. TCFG 的 En、Periodic 和 InitVal 域

TCFG 中各个域的更新维护比较简单, 其复杂性体现在各个域对 timer_cnt 的计数控制上, 这些将在下面讲述 TVAL 时说明。这里给出 TCFG 自身的示例 Verilog 如下:

```
always @(posedge clock) begin
    if (reset)
        csr_tcfg_en <= 1'b0;
    else if (csr_we && csr_num==`CSR_TCFG)
        csr_tcfg_en <= csr_wmask[`CSR_TCFG_EN]&csr_wvalue[`CSR_TCFG_EN]
            | ~csr_wmask[`CSR_TCFG_EN]&csr_tcfg_en;

    if (csr_we && csr_num==`CSR_TCFG) begin
        csr_tcfg_periodic <= csr_wmask[`CSR_TCFG_PERIOD]&csr_wvalue[`CSR_TCFG_
            PERIOD]
            | ~csr_wmask[`CSR_TCFG_PERIOD]&csr_tcfg_periodic;
        csr_tcfg_initval <= csr_wmask[`CSR_TCFG_INITV]&csr_wvalue[`CSR_TCFG_
            INITV]
    end
end
```

```

| ~csr_wmask[`CSR_TCFG_INITV]&csr_tcfg_initval;
end
end

```

14. TVAL 的 TimeVal 域

TVAL 的 TimeVal 域是一个软件只读域，它返回定时器计数器的值，所以可以将其实现为 wire 而非 reg。此处的设计关键点在于用作定时器的计数器 timer_cnt 的实现。这里先给出 Verilog 代码实现如下：

```

reg          csr_tcfg_en;
reg          csr_tcfg_periodic;
reg [29:0]   csr_tcfg_initval;
wire [31:0]  tcfg_next_value;
wire [31:0]  csr_tval;
reg [31:0]   timer_cnt;

assign tcfg_next_value =  csr_wmask[31:0]&csr_wvalue[31:0]
                        | ~csr_wmask[31:0]&{csr_tcfg_initval,
                                           csr_tcfg_periodic, csr_tcfg_en};

always @(posedge clock) begin
    if (reset)
        timer_cnt <= 32'hffffffff;
    else if (csr_we && csr_num==`CSR_TCFG && tcfg_next_value[`CSR_TCFG_EN])
        timer_cnt <= {tcfg_next_value[`CSR_TCFG_INITVAL], 2'b0};
    else if (csr_tcfg_en && timer_cnt!=32'hffffffff) begin
        if (timer_cnt[31:0]==32'b0 && csr_tcfg_periodic)
            timer_cnt <= {csr_tcfg_initval, 2'b0};
        else
            timer_cnt <= timer_cnt - 1'b1;
    end
end

assign csr_tval = timer_cnt[31:0];

```

上面的代码可能有两处理解起来有点难度，解释一下：

- 1) 我们在软件对 timer 进行配置（也就是更新 TCFG 控制状态寄存器的时候）的同时发起 timer_cnt 的更新操作。具体来说，就是当软件开启 timer 的使能时（即将 TCFG 的 En 域置 1），将此时写入的 timer 配置寄存器的定时器初始值更新到 timer_cnt 中；当软件关闭 timer 的使能时，timer_cnt 不更新。因为是在软件写 TCFG 的同时更新 timer，所以要看当前写入 TCFG 寄存器的值“tcfg_next_value”，而不是用 TCFG 寄存器已有的值。
- 2) 当 timer_cnt 减到全 0 且定时器不是周期性工作模式的情况下，代码中没有专门处理的逻辑，所以 timer_cnt 会继续减 1 变成 32'hffffffff。不过，因为此时定时

器是非周期性的, 所以它应该停止计数, 这就是为何 timer_cnt 自减的使能条件除了看 csr_tcfg_en 是否为 1 外还会看 “timer_cnt!=32'hffffffff” 这个条件。

15. TICLR 的 CLR 域

TICLR 的 CLR 域很特殊, 它的读写属性是 “W1”, 意味着软件只有对它写 1 才会产生执行效果 (即硬件只捕获对 TICLR 的 CLR 域写 1 这个动作), 写 0 无效, 同时软件读出的值永远是 0。所以, TICLR 的 CLR 域并不需要定义一个 reg 与之对应, 我们只需要定义一个恒为 0 的 wire 用于后面的 CSR 读出即可。

```
assign csr_ticlr_clr = 1'b0;
```

16. CSR 的读出逻辑

前面为了代码的可读性和易维护性, 将 CSR 的各个域拆分后实现其更新维护逻辑, 不过在 CSR 指令读取值的时候, 每个 CSR 需要重新拼合出一个 32 位宽的值返回。以下给出 Verilog 代码片段示意:

```
wire [31:0] csr_crmd_rvalue = {23'b0, csr_crmd_datm, csr_crmd_datf, csr_crmd_
pg, csr_crmd_da, csr_
wire [31:0] csr_prmd_rvalue = {29'b0, csr_prmd_pie, csr_prmd_pplv};
wire [31:0] csr_ecfg_rvalue = {19'b0, csr_ecfg_lie};
.....
assign csr_rvalue = {32{csr_num==`CSR_CRMD}} & csr_crmd_rvalue
                    | {32{csr_num==`CSR_PRMD}} & csr_prmd_rvalue
                    | {32{csr_num==`CSR_ECFG}} & csr_ecfg_rvalue
                    .....
```

7.3.4 处理控制状态寄存器相关引发的冲突

指令手册 7.3 节提到了 “控制状态寄存器相关所引发的冲突” 这样一个概念, 并且说在 LoongArch 指令系统中, 由硬件来负责维护。由于这个概念在一般的教科书中较少涉及, 我们在这里进行一些解释。

前面在介绍流水线 CPU 设计的时候, 曾经专门分析过围绕通用寄存器的数据相关以及由此而产生的数据相关冲突。这里将要分析的控制状态寄存器相关引发的冲突与之类似, 它是由围绕 CSR 的 “写后读” 相关引起的, 因 CPU 采用了多级流水线结构设计而显现出来。

最典型的 CSR 写后读相关是 csrwr 或 csrchg 指令修改一个 CSR 后又有 csrrd、csrwr 或 csrchg 指令读取同一个 CSR。为了解决这个相关所引起的冲突, 最简单有效

的方式是将所有 CSR 读写指令访问 CSR 的操作放到同一级流水线处理。这种解决方案有效且实现简单，缺点是会造成一定的性能损失。这里性能损失的原因是，我们如果将 CSR 指令写 CSR 的动作放到写回级处理，那么后续将 CSR 指令返回值作为源操作数的指令与 CSR 指令之间就存在 2 拍的执行延迟，即读取 CSR 指令返回值的第二条指令必须将自身阻塞在译码级直至前面的 CSR 指令到达写回级。那么我们为什么不用前递的方式来解决这种写后读相关引发的冲突呢？主要是因为 CSR 指令是特权指令，只有内核之类的特权软件中才使用，而且即使用也并不多见写入一个 CSR 立即又要将其读出来的情况。对于这种小概率场景进行性能优化，对整体性能的提升作用微乎其微，投入产出比太低，因此我们并不考虑。

除了上面 CSR 指令之间的写后读相关外，还有其他类型的 CSR 写后读相关。通过前面 7.3.3 节的分析，我们已知 CSR 的各个域有不同的读者、写者。这些存在写后读相关的写者和读者，只要不是在同一级流水级，就会引发冲突。主要有如表 7.1 所示的四种情况。

表 7.1 其他可能引发冲突的控制状态寄存器相关情况

序号	写者	相关对象	读者
1	csrwr 或 csrxchg	CRMD.IE、ECFG.LIE、ECFG.IS[1:0]、TCFG.En、TICLR.CLR	译码级的指令（标记中断）
2	csrwr 或 csrxchg	ERA、PRMD.PPLV、PRMD.PIE	ertn
3	ertn	CRMD.IE	译码级的指令（标记中断）
4	ertn	CRMD.PLV	取指的 PC

解决相关所引发的冲突，思路无外乎“阻塞 + 前递”。我们这里仍然不引入前递，只单纯地利用阻塞来解决。表 7.1 中提到的前面三种情况用阻塞来解决的实现方式很直接，即判断执行、访存、写回级有没有这几种情况中的写相关对象的写者，如果有就把读者阻塞在译码级且什么都不做（什么都不做意味着不要进行中断标记，不要修改取指 PC）。最大的实现困难似乎体现在表 7.1 中的第 4 种情况，因为流水线中最早只有在译码级才可以知晓写者和相关对象，但是此时取指级很可能已经有一条指令了，pre-IF 级也很可能把不合适的 PC 取指请求发出去了^①，也就是说单纯靠阻塞是没办法彻底解决问题的。为此，我们引入一种特殊的解决方案：ertn 指令直到写回级才修改 CRMD，与此同时清空流水线并更新取指 PC。这也就是前面提到的 ertn_flush 信号的由来。对于流水级缓存来说，这个信号与异常信号 wb_ex 的作用是一样的，所不同的是它不是一个软件可见的异常。为什么这种方案能解决表 7.1 中的第 4 种情况呢？请读者思考一下。

① 初学者对这句话可能很难理解，请等完成 AXI 总线接口实现的学习实践后再来理解这一句。

7.4 其他指令的实现

在实现了定时器中断的支持后，我们在这一部分再实现 3 条计时器相关的指令：`rdcntvl.w`、`rdcntvh.w` 和 `rdcntid`。这里 `rdcntvl.w` 和 `rdcntvh.w` 两条指令分别读取计时器的低 32 位和高 32 位值写入到第 `rd` 项寄存器中。这里的计时器通过一个 64 位的计数器实现，复位为 0，复位结束后每个时钟周期自增 1，且该计数器软件无法修改，只能通过 `rdcntvl.w` 和 `rdcntvh.w` 指令读取。它是一个独立的计数器，不是产生定时器中断时所用的那个倒计时计数器。不过，这两个计数器所用的时钟是同一个恒定频率的时钟。本书实践任务涉及范围内，可以不考虑这个恒定频率的时钟如何实现，直接使用流水线的时钟。`rdcntid` 指令读取的就是 TID 控制状态寄存器中的内容。

由于 64 位的计时器并不会被软件修改，所以 `rdcntvl.w` 和 `rdcntvh.w` 指令在执行、访存、写回级读取它的值都是没有问题的，我们推荐在执行流水级读取，可以减少不需要的阻塞。`rdcntid` 指令所读的 TID 控制状态寄存器可以被 CSR 指令修改，所以简单起见可以将其读取 CSR 的操作推迟到写回级进行。

另外需要说明的是，LoongArch32 精简版中“`rdcntvl.w rd`”“`rdcntvh.w rd`”和“`rdcntid rj`”三条指令的编码分别对应 LoongArch 指令集中的“`rdtimel.w rd, zero`”“`rdtimeh.w rd, zero`”和“`rdtimel.w zero, rj`”，因此在反汇编中你将看到对应 LoongArch 指令集的汇编助记形式。

7.5 任务与实践

完成本章的学习后，希望读者能够完成以下 2 个实践任务：

- 1) 添加系统调用异常支持，参见 7.5.1 节。
- 2) 添加其他异常与中断支持，参见 7.5.2 节。

7.5.1 实践任务 12：添加系统调用异常支持

本实践任务要求在实践任务 11 实现的 CPU 基础上完成以下工作：

- 1) 为 CPU 增加 `csrwr`、`csrrd`、`csrxchg` 和 `ertn` 指令。
- 2) 为 CPU 增加控制状态寄存器 `CRMD`、`PRMD`、`ESTAT`、`ERA`、`EENTRY`、`SAVE0` ~ 3。
- 3) 为 CPU 增加 `syscall` 指令，实现系统调用异常支持。
- 4) 运行 `exp12` 对应的 `func`，要求成功通过仿真和上板验证。

请参照 2.3.1 节中介绍的方式获取本次实践任务所需的实验开发环境。具体的实验环境仍位于 mycpu_env/ 目录下，且仍使用 soc_bram/ 子目录。

实验环境准备就绪后，请参考下列步骤完成本实践任务：

- 1) 将所实现 CPU 的代码更新至 mycpu_env/myCPU/ 目录中。
- 2) 修改 func 配置文件——mycpu_env/func/include/test_config.h，选择 exp12 的配置，编译。（如果是通过压缩包 exp12.zip 获取实验开发环境的，请跳过该步骤。）
- 3) 打开 gettrace 工程——mycpu_env/gettrace/gettrace.xpr。（该 Vivado 工程中的 IP 核是使用 Vivado 2019.2 创建的，如果使用更高版本的 Vivado 打开，请参考附录 D.4 节进行 IP 核升级。）运行 gettrace 工程的仿真（进入仿真界面后，直接点击 run all 等待仿真运行完成），生成新的参考 trace 文件 golden_trace.txt（mycpu_env/gettrace/golden_trace.txt）。要等仿真运行完成，golden_trace.txt 才有完整的内容。（如果是通过压缩包 exp12.zip 获取实验开发环境的，请跳过该步骤。）
- 4) 进入 mycpu_env/soc_verify/soc_bram/run_vivado/ 目录下启动验证 myCPU 的工程。如果该目录下尚未创建工程，请参考附录 D.2 节介绍的步骤，利用该目录下的 create_project.tcl 文件创建工程。如需要，请参考附录 D.4 节进行 IP 核升级。如果该目录下已有前一实践任务创建过的工程，可以在打开工程后，参照附录 D.3 节介绍的步骤，更新项目中 CPU 实现文件的列表。
- 5) 参考 4.4.5.2 节，对工程中的 inst_ram 重新定制。（如果是通过压缩包 exp12.zip 获取实验开发环境的，请跳过该步骤。）
- 6) 在验证 myCPU 的工程中运行仿真（进入仿真界面后，直接点击 run all），进行功能验证与调试，直至仿真测试通过。
- 7) 在验证 myCPU 的工程中综合实现后生成二进制码流文件，进行上板验证。（如无硬件实验平台，请跳过该步骤。）

7.5.2 实践任务 13：添加其他异常与中断支持

本实践任务要求在实践任务 12 实现的 CPU 基础上完成以下工作：

- 1) 为 CPU 增加取指地址错（ADEF）、地址非对齐（ALE）、断点（BRK）和指令不存在（INE）异常的支持。
- 2) 为 CPU 增加中断的支持，包括 2 个软件中断、8 个硬件中断和定时器中断。
- 3) 为 CPU 增加控制状态寄存器 ECFG、BADV、TID、TCFG、TVAL、TICLR。

4) 为 CPU 增加 `rdcntvl.w`、`rdcntvh.w` 和 `rdcntid` 指令。

5) 运行 `exp13` 对应的 `func`，要求成功通过仿真和上板验证。

请参照 2.3.1 节中介绍的方式获取本次实践任务所需的实验开发环境。具体的实验环境仍位于 `mycpu_env/` 目录下，且仍使用 `soc_bram/` 子目录。

实验环境准备就绪后，请参考下列步骤完成本实践任务：

- 1) 将所实现 CPU 的代码更新至 `mycpu_env/myCPU/` 目录中。
- 2) 修改 `func` 配置文件——`mycpu_env/func/include/test_config.h`，选择 `exp13` 的配置，编译。（如果是通过压缩包 `exp13.zip` 获取实验开发环境的，请跳过该步骤。）
- 3) 打开 `gettrace` 工程——`mycpu_env/gettrace/gettrace.xpr`。（该 Vivado 工程中的 IP 核是使用 Vivado 2019.2 创建的，如果使用更高版本的 Vivado 打开，请参考附录 D.4 节进行 IP 核升级。）运行 `gettrace` 工程的仿真（进入仿真界面后，直接点击 `run all` 等待仿真运行完成），生成新的参考 `trace` 文件 `golden_trace.txt`（`mycpu_env/gettrace/golden_trace.txt`）。要等仿真运行完成，`golden_trace.txt` 才有完整的内容。（如果是通过压缩包 `exp13.zip` 获取实验开发环境的，请跳过该步骤。）
- 4) 进入 `mycpu_env/soc_verify/soc_bram/run_vivado/` 目录下启动验证 `myCPU` 的工程。如果该目录下尚未创建工程，请参考附录 D.2 节介绍的步骤，利用该目录下的 `create_project.tcl` 文件创建工程。如需要，请参考附录 D.4 节进行 IP 核升级。如果该目录下已有前一实践任务创建过的工程，可以在打开工程后，参照附录 D.3 节介绍的步骤，更新项目中 CPU 实现文件的列表。
- 5) 参考 4.4.5.2 节，对工程中的 `inst_ram` 重新定制。（如果是通过压缩包 `exp13.zip` 获取实验开发环境的，请跳过该步骤。）
- 6) 在验证 `myCPU` 的工程中运行仿真（进入仿真界面后，直接点击 `run all`），进行功能验证与调试，直至仿真测试通过。
- 7) 在验证 `myCPU` 的工程中综合实现后生成二进制码流文件，进行上板验证。（若无硬件实验平台，请跳过该步骤。）